

Università degli Studi di Messina

Department of Mathematics and Computer Sciences

Parallel Image Processing Pipeline

Development of a Hybrid MPI–OpenMP–OpenCL Parallel Image Processing Pipeline with User-Customisable Workflows Using FFT and Frequency-Domain Analysis

Student
Bahati Patrick
Matricola: 573868

Supervisor
Prof. Salvatore Distefano

Course: High Performance Computing

Academic Year 2025–2026

Abstract

This report presents the design, implementation, and evaluation of a hybrid parallel image-processing pipeline combining MPI, OpenCL, and OpenMP, applied to frequency-domain feature extraction on a dataset derived from the EuroSAT remote-sensing benchmark, of $N = 54{,}597$ image files: the benchmark's 27,000 patches in both RGB and multispectral form, plus 597 further multispectral files. The files are partitioned across MPI ranks (data parallelism); within each rank, a three-thread queue overlaps per-batch disk I/O, GPU computation, and CPU post-processing (task/pipeline parallelism); OpenCL work-items and an OpenMP reduction provide fine-grained data parallelism inside the GPU kernels and the feature-extraction stage respectively.

On a single-node, 10-core Apple Silicon testbed (one integrated GPU), strong scaling reaches a peak throughput of 10,170 images/second at $P = 8$ (speedup 3.17×, efficiency 39.7 %; 92.9 % at $P = 2$; the speedup at $P = 8$ ranged from 2.2× to 3.2× across repeat sessions, depending on the file-cache state), and the hybrid pipeline is 1.43× faster than a single-threaded sequential baseline at $P = 1$ and 4.49× at $P = 8$. Per-stage instrumentation attributes the sublinear scaling primarily to load imbalance: from $P = 4$ the contiguous partition gives some ranks mainly RGB JPEGs and others mainly 13-band GeoTIFFs, which take more than twice as long to load, so the JPEG ranks finish early and wait. Contention for the shared GPU is a secondary factor. An Amdahl's Law fit gives an effective non-scaling fraction $f \approx 0.22$. The practical value of task parallelism is quantified directly: at $P = 8$, pipelined execution is 2.3× faster end to end than sequential-stage execution, of which stage overlap explains at most about 1.9× and a file-cache effect the rest. Inter-process communication is negligible, reduced by more than 12,000× through local feature reduction before the `MPI_Gather` of the features.

Contents

Abstract **1**

1 Introduction **3**
1.1 Context and Motivation . . . 3
1.2 Problem Statement . . . 3
1.3 Contributions . . . 4
1.4 System Architecture Overview . . . 4
1.5 Report Organisation . . . 4
1.6 Related Work . . . 5

2 Problem Rationale **6**
2.1 Formal Problem Statement . . . 6
2.2 Sequential Baseline . . . 7
2.3 Background Technologies . . . 8

3 Decomposition, Partitioning, and Pipeline Design **9**
3.1 Data Decomposition: Static Block Distribution . . . 9
3.2 Task Decomposition and Stage Dependency Graph . . . 10
3.3 Task Parallelism: Three-Thread Intra-Rank Pipeline . . . 11
3.4 User-Customisable Workflow . . . 14

4 Communication, Aggregation, and Agglomeration **15**
4.1 Interaction Graph and Communication Costs . . . 15
4.2 Feature Aggregation Strategy . . . 16
4.3 Agglomeration . . . 16

5 Performance Evaluation **18**
5.1 Testbed and Hardware Mapping . . . 18
5.2 Experimental Methodology . . . 19
5.3 Strong Scaling . . . 20
5.4 Weak Scaling . . . 22
5.5 GPU Concurrency Experiment . . . 23
5.6 Additional Experiments . . . 26
5.7 Summary and Comparative Benchmarking . . . 29

6 Conclusions **30**
6.1 Summary . . . 30
6.2 Limitations and Future Work . . . 30

References **31**

CHAPTER 1

Introduction

1.1 Context and Motivation

Earth-observation missions generate large volumes of imagery: the Sentinel-2 constellation revisits any given land surface every five days at spatial resolutions between 10 and 60 metres [5]. A sequential pipeline, which completes the full processing chain for one image before beginning the next, does not scale to such data. High Performance Computing architectures address this by combining distributed-memory parallelism, shared-memory parallelism and accelerators. The Fast Fourier Transform (FFT) [4], with its $\mathcal{O}(n \log n)$ complexity for n samples, is a canonical frequency-domain kernel and is well suited to GPU acceleration.

1.2 Problem Statement

This project designs, implements, and evaluates a hybrid parallel image-processing pipeline that applies frequency-domain filtering and statistical feature extraction to a locally assembled dataset derived from the EuroSAT remote-sensing benchmark [10]. The dataset contains 54,597 image files: 27,000 RGB JPEGs and their corresponding 27,000 13-band multispectral GeoTIFFs, together with 597 additional multispectral GeoTIFFs. The original EuroSAT benchmark consists of 27,000 labelled, georeferenced Sentinel-2 patches of 64×64 pixels across 10 land-cover classes. The additional files are not part of these original benchmark patches (`experiments/verify_dataset.py`). The RGB and multispectral versions represent the same patches, so the 54,597 files do not correspond to 54,597 independent observations. No classification is performed; each file is treated as a separate processing task. Multispectral images are reduced to their true-colour bands so that all images enter the processing pipeline in the same three-channel format.

The pipeline combines three levels of parallelism:

1. **MPI data parallelism:** P processes execute the same four-stage pipeline on disjoint subsets of the image files.

2. **Task/pipeline parallelism:** a three-thread intra-rank producer–consumer

queue overlaps disk I/O, GPU computation, and CPU post-processing on successive batches.

3. **Fine-grained data parallelism:** OpenCL work-items execute the GPU FFT and filtering operations, while OpenMP threads parallelise CPU feature extraction.

1.3 Contributions

The principal contributions of this work are:

1. a hybrid MPI + OpenCL + OpenMP pipeline with two GPU kernels, a radix-2 2D FFT verified against NumPy and a high-pass filter;
2. a three-thread intra-rank pipeline whose task parallelism is demonstrated by per-batch stage timing (Table 5.9);
3. a YAML-driven DAG, validated with a topological sort, that reconfigures the workflow without recompilation;
4. a performance evaluation covering strong and weak scaling, GPU concurrency, OpenMP threads and a per-stage breakdown.

1.4 System Architecture Overview

Figure 1.1 shows how the Python orchestration layer, the MPI processes and the OpenCL kernel library interact across the four stages.

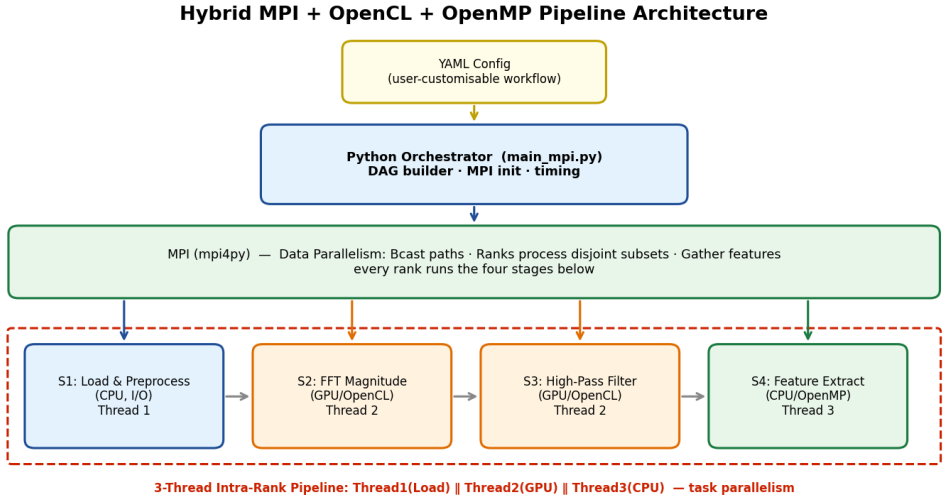


Figure 1.1: System architecture of the hybrid MPI + OpenCL + OpenMP pipeline.

1.5 Report Organisation

Chapter 2 states the formal problem and the sequential baseline. The design then follows Foster's four PCAM steps in order: partitioning and the task-parallel pipeline

in Chapter 3, communication and agglomeration in Chapter 4, and mapping at the start of Chapter 5, which then presents the experiments. Chapter 6 concludes. The source code, experiment scripts and raw results are available at https://github.com/bahatipatrick87/HpcFinalProject.

1.6 Related Work

1.6.1 Parallel FFT Libraries

FFTW [7] is a widely used library for computing discrete Fourier transforms, providing optimised CPU implementations and support for both shared-memory and distributed-memory execution through its MPI interface [8]. On the GPU side, clFFT [1] provides an OpenCL-based library for performing fast Fourier transforms on supported GPU platforms.

Unlike these general-purpose libraries, the present project implements a radix-2 two-dimensional FFT directly using OpenCL kernels. This approach provides control over the GPU computation and its integration with the image-processing pipeline, while numerical results are validated against a NumPy reference.

1.6.2 Hybrid MPI and Accelerator Programming

Hybrid parallel programming combines distributed-memory communication through MPI with computation performed by accelerator technologies such as CUDA and OpenCL. This approach has been studied in applications including stencil computations, linear algebra, and particle simulations [14].

The present project applies this hybrid approach to an end-to-end image-processing pipeline. MPI distributes image-file workloads across processes, while OpenCL performs GPU-based FFT and filtering operations and OpenMP parallelises CPU feature extraction. In addition, a three-thread producer–consumer pipeline overlaps disk loading, GPU computation, and CPU post-processing.

1.6.3 Remote-Sensing Platforms

Google Earth Engine [9] is a cloud-based platform for large-scale geospatial data processing and analysis. It provides managed access to satellite imagery and distributed computing capabilities, making it suitable for remote-sensing applications. However, it offers a different programming model from the explicit process-level and accelerator-level control used in this project. The present work focuses on a locally executed, single-node heterogeneous pipeline for batch processing of remote-sensing images. It combines MPI, OpenCL, and OpenMP to investigate workload distribution, GPU computation, pipeline overlap, and communication costs when extracting frequency-domain features from satellite images.

CHAPTER 2

Problem Rationale

2.1 Formal Problem Statement

> *Given the EuroSAT dataset of Section 1.2, $\mathcal{D} = \{I_i\}_{i=1}^{N}$ with $N = 54{,}597$ and each loaded as $I_i \in \mathbb{R}^{64\times64\times3}$, design and implement a hybrid parallel pipeline Π that performs frequency-domain filtering and statistical feature extraction on heterogeneous HPC hardware, minimising total wall-clock time T_{total} while maintaining correct results, strong-scaling efficiency, and runtime reconfigurability.*

2.1.1 Mathematical Formulation

The pipeline $\Pi = (S_1, S_2, S_3, S_4)$ processes each image through four sequentially dependent stages:

$$I'_i = \text{Load}(I_i) \in [0,1]^{64\times64\times3} \quad \text{(decode and normalise)}, \tag{2.1}$$

$$\hat{I}_i = \left|\mathcal{F}\left(I'_i\right)\right| \quad \text{(2D magnitude spectrum)}, \tag{2.2}$$

$$\tilde{I}_i = \hat{I}_i \circ H \quad \text{(element-wise high-pass filter)}, \tag{2.3}$$

$$f_i = \frac{1}{|\tilde{I}_i|} \sum_{u,v,c} \tilde{I}_i(u,v,c) \in \mathbb{R}, \tag{2.4}$$

where Load divides JPEG pixels by 255 and, for GeoTIFFs, keeps the true-colour bands 4/3/2 and divides the 16-bit reflectance by 2000 (clipped to $[0,1]$); $\mathcal{F}$ denotes the 2D Discrete Fourier Transform of each channel, $|\cdot|$ the element-wise modulus, and $\circ$ the element-wise (Hadamard) product. The high-pass mask, applied to each channel, is $H(u,v) = 1 - \exp\!\left(-d(u,v)^2/50\right)$, where $u \in \{0,\ldots,W-1\}$ and $v \in \{0,\ldots,H_{\text{img}}-1\}$ are the DFT frequency indices with the DC component at $(0,0)$ and no spectrum shift, and $d(u,v)^2 = \min(u, W-u)^2 + \min(v, H_{\text{img}}-v)^2$ is the squared wrap-around distance from DC, for image width W and height H_{img} (both 64).

Both the sequential reference (Section 2.2) and the GPU `fft_magnitude` kernel compute $|\mathcal{F}|$: the kernel is a radix-2 Cooley–Tukey FFT applied to the rows and

then the columns of each channel. `tests/test_correctness.py` checks it against `numpy.fft.fft2`, and `experiments/verify_fft_against_numpy.py` confirms that the features of all 54,597 images agree with a NumPy computation to within 10^{-6} (relative) at $P = 1, 2, 4, 8$.

2.1.2 Performance Objectives

The following quantitative objectives guided the design; their measured outcomes are reported in Chapter 5:

- **Throughput:** exceed 1,000 images/s at $P = 8$.
- **Communication efficiency:** keep the `MPI_Gather` payload below 1 MB.
- **Pipeline overlap:** confirm overlap through per-batch stage timing.
- **Strong scaling:** anticipate sublinear scaling due to shared-resource contention (GPU and CPU), analysed in Section 5.3.1 and quantified using Amdahl's Law [2] in Section 5.3.3.

2.2 Sequential Baseline

The sequential baseline processes each image completely through all four stages before advancing to the next (Algorithm 1, Figure 2.1), where M is the number of pixels per image, as distinct from the number of files N. It serves as the reference for the speedups of Section 5.7.

Algorithm 1 Sequential image-processing pipeline.

Require: dataset path $\mathcal{P}$
Ensure: feature vector $\mathbf{v}$, one scalar feature per image
$\mathcal{L} \leftarrow \textsc{LoadImageList}(\mathcal{P})$
$\mathbf{v} \leftarrow [\,]$
for each path $p_i \in \mathcal{L}$ **do**
  $I_i \leftarrow \textsc{ReadImage}(p_i)$ {disk I/O, single thread}
  $I_i' \leftarrow \textsc{Normalise}(I_i)$ {to $[0, 1]$, Eq. (2.1)}
  $\hat{I}_i \leftarrow |\mathcal{F}(I_i')|$ {$\mathcal{O}(M \log M)$, CPU}
  $\tilde{I}_i \leftarrow \hat{I}_i \circ H$ {$\mathcal{O}(M)$, CPU}
  $f_i \leftarrow \text{mean}(\tilde{I}_i)$ {$\mathcal{O}(M)$, CPU}
  $\mathbf{v}$.append(f_i)
end for
return $\mathbf{v}$

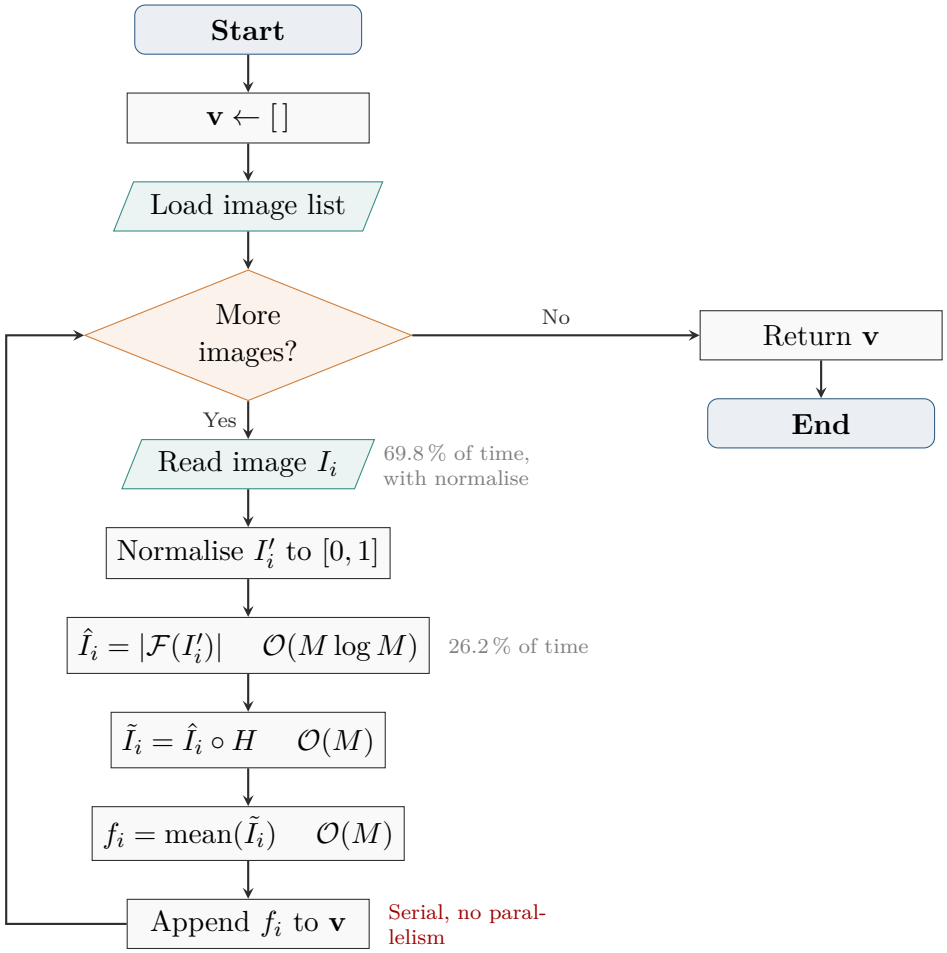


Figure 2.1: Sequential baseline flowchart. Each image traverses all four stages before processing begins on the next. Total time: $T_{\text{seq}} = \sum_{i=1}^{N} (t_{\text{load}}^{(i)} + t_{\text{FFT}}^{(i)} + t_{\text{filt}}^{(i)} + t_{\text{feat}}^{(i)})$.

2.2.1 Bottleneck Analysis

The sequential implementation executes all stages serially and does not use the GPU or overlap computation. Profiling the full dataset (`experiments/run_sequential_baseline.py`, mean of three trials) gives the following time distribution: loading 69.8 %, FFT 26.2 %, filtering 2.9 %, and feature extraction 1.2 %. Loading and FFT together account for approximately 96 % of execution time, making them the main optimisation targets. The single-threaded baseline uses only one CPU core and serves as the reference for evaluating the hybrid implementations.

2.3 Background Technologies

MPI [12] provides distributed-memory communication between processes. OpenCL [15], a vendor-neutral standard, was chosen over CUDA for compatibility with the Apple Silicon GPU; Stages 2 and 3 are OpenCL C kernels embedded in the C++ shared library. OpenMP [13] provides shared-memory parallelism through compiler directives, applied to Stage 4.

CHAPTER 3

Decomposition, Partitioning, and Pipeline Design

The design follows Foster's PCAM methodology [6], whose four steps structure this and the next two chapters: **P**artitioning into images and array elements, with the stage DAG (Sections 3.1–3.2); **C**ommunication (Sections 4.1–4.2); **A**gglomeration into per-rank blocks and 256-image batches (Section 4.3); and **M**apping onto processes, the GPU and CPU cores (Section 5.1). Data parallelism comes from the partition across ranks (Section 3.1) and task parallelism from the three-thread pipeline over the stages (Section 3.3).

3.1 Data Decomposition: Static Block Distribution

The dataset is partitioned across P MPI ranks using a balanced static block distribution. Let $Q = \lfloor N/P \rfloor$ and $R = N \bmod P$. Rank $r \in \{0, \ldots, P-1\}$ is assigned images with indices in $[s_r,\, s_r + n_r)$, where

$$s_r = rQ + \min(r, R), \qquad n_r = \begin{cases} Q+1 & r < R, \\ Q & r \geq R. \end{cases} \tag{3.1}$$

For $N = 54{,}597$ and $P = 8$: $Q = 6{,}824$, $R = 5$; ranks 0–4 receive 6,825 images and ranks 5–7 receive 6,824.

```
all_files = comm.bcast(all_files, root=0)  # MPI_Bcast: same list on all ranks
q, r  = len(all_files) // size, len(all_files) % size
start = rank * q + min(rank, r)                    # Eq. (3.1)
end   = start + q + (1 if rank < r else 0)
my_files = all_files[start:end]  # same code, different data (SPMD)
# ... each rank runs S1-S4 on its own my_files, no messages ...
all_lists     = comm.gather(local_list, root=0)     # 1 float/image
all_durations = comm.gather(local_duration, root=0) # T_P = max(.)
```

Listing 3.1: MPI data parallelism, simplified from `dataset_loader.py` and `main_mpi.py`.

3.2 Task Decomposition and Stage Dependency Graph

3.2.1 Stage Dependency DAG (Single Rank)

Figure 3.1 is a Directed Acyclic Graph (DAG) whose nodes are the four pipeline stages and whose directed edges represent data dependencies. For one batch on one rank, stages execute strictly left to right. Node shading encodes the target device (teal = CPU; orange = GPU).

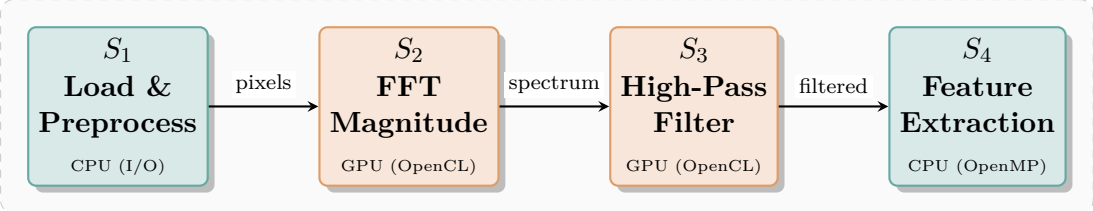


Figure 3.1: Stage dependency DAG for one batch on a single MPI rank. Nodes are stages; edges are data dependencies.

3.2.2 MPI Data Distribution

Figure 3.2 illustrates how the dataset is partitioned across P ranks. There are **no peer-to-peer MPI messages**. The only MPI operations are collectives (Chapter 4): `MPI_Barrier` before timing, `MPI_Bcast` of the file list at startup, and two `MPI_Gather` calls at the end, for the features and the per-rank times.

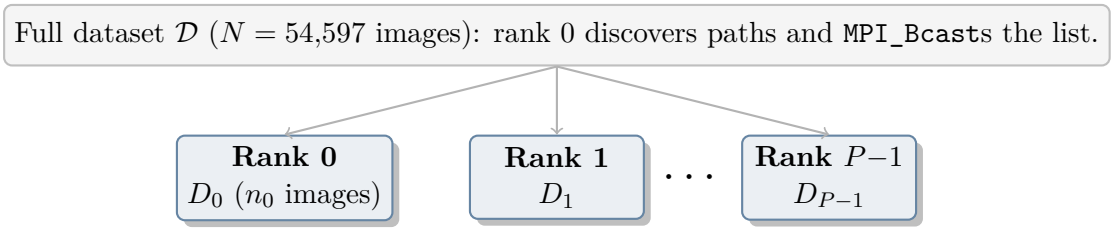


Figure 3.2: MPI data parallelism: the full dataset is broadcast and each rank retains a contiguous block of paths (Equation (3.1)), with $D_0 \cup \cdots \cup D_{P-1} = \mathcal{D}$ and the blocks disjoint. Arrows show partition ownership, not a processing pipeline between ranks.

3.2.3 Per-Rank Stage Chain

Figure 3.3 confirms that every rank runs the same four-stage chain on its local data. This is replicated-pipeline data parallelism, not rank specialisation (where different ranks would handle different stages).

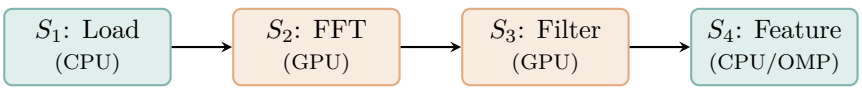


Figure 3.3: Per-rank stage chain: every rank applies the same four stages to each batch of its own subset, which is data parallelism. Edges are intra-rank data dependencies; there is no inter-rank compute flow.

3.3 Task Parallelism: Three-Thread Intra-Rank Pipeline

Task parallelism is implemented as a three-thread queue pipeline inside each rank.

3.3.1 Sequential vs. Pipeline Execution

Sequential mode (`task_parallel_prefetch: false`). Each rank loads its full subset, runs FFT and filter on it, then extracts features, so stages do not overlap. This mode exists for comparison.

Pipeline mode (`task_parallel_prefetch: true`, default). Images are processed in chunks of size `io_batch_size`, and three threads per rank form a bounded producer–consumer queue (Listing 3.2): Thread 1 loads and normalises batch $k{+}1$, Thread 2 applies the two OpenCL kernels to batch k, and Thread 3 runs the OpenMP feature reduction on batch $k{-}1$. These different operations overlap in wall-clock time on different batches, which is pipeline/task parallelism. Bounded queues provide back-pressure, a sentinel propagates shutdown, and the threads overlap because the GIL is released during disk I/O and every `ctypes` call.

```
q1, q2 = queue.Queue(maxsize=3), queue.Queue(maxsize=3)  # bounded queues
def io_worker():                 # Thread 1: load batch k+1
    for chunk in chunks:
        ctx = context.copy(); ctx["input_data"] = load_chunk(chunk)
        q1.put(ctx)
    q1.put(SENTINEL)
def compute_worker():            # Thread 2: GPU stages on batch k
    while (ctx := q1.get()) is not SENTINEL:
        fft_frequency_stage(ctx) # OpenCL fft_magnitude
        filter_stage(ctx)        # OpenCL highpass_filter
        q2.put(ctx)
    q2.put(SENTINEL)
def feature_worker():            # Thread 3: OpenMP features, batch k-1
    while (ctx := q2.get()) is not SENTINEL:
        feature_extraction_stage(ctx)
        all_features.append(ctx["features"])
for t in (t1, t2, t3): t.start() # all three stages run concurrently
for t in (t1, t2, t3): t.join()
```

Listing 3.2: Three-thread task-parallel pipeline, simplified from `batched_pipeline.py`.

Design choice. An alternative is to express the stages as OpenMP tasks with asynchronous OpenCL events inside the C++ library. Python threads with bounded queues were chosen because loading runs in Python (Pillow, `tifffile`) and the stages are configured and orchestrated there: each thread makes blocking calls, yet the stages overlap

on successive batches, and the queues bound memory use.

3.3.2 Why Intra-Rank Threads Rather Than Dedicated MPI Ranks

An inter-rank stage pipeline, in which rank 0 only loads data and passes it to rank 1 for the FFT, is a valid textbook design. The intra-rank pipeline is preferred because such a design would send about 12.6 MB per batch and stage as MPI messages, where the present design gathers one float per image; because a rank specialised to the FFT would still contend for the same single GPU; and because images are independent items suited to MPI distribution, while threads suit overlapping stages within one rank's batch stream.

3.3.3 Gantt Diagram and Timing Evidence

Figure 3.4 shows the schedule of the three threads; Table 5.9 gives the measured stage times.

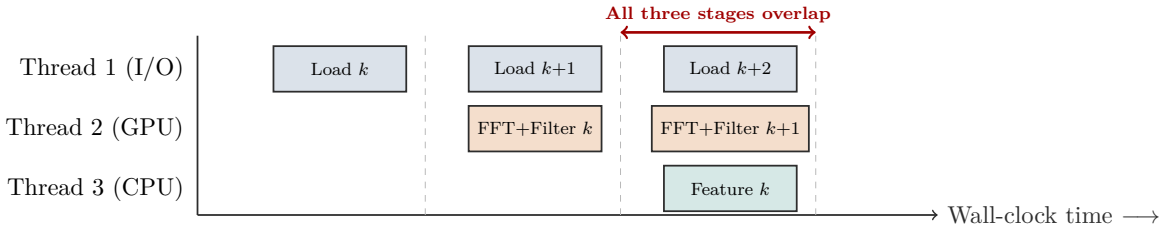


Figure 3.4: **Task parallelism via pipeline:** three threads stagger Load (I/O), FFT+Filter (GPU), and Feature (CPU) across successive batches k, k+1, k+2. From the second dashed clock-tick line onwards all three threads are busy at once, each on a different batch (schematic; bar lengths are not to scale, measured stage times are in Table 5.9). Implemented with two bounded `queue.Queue` objects in `src/pipeline/batched_pipeline.py`.

3.3.4 Parallelism Taxonomy

Table 3.1 classifies the parallelism mechanisms of the implementation.

Table 3.1: Classification of parallelism mechanisms in the hybrid pipeline.

Granularity	Mechanism	Parallelism type
Across MPI ranks	Same pipeline on disjoint image subsets	**Data parallelism** (primary)
Across successive batches	3-thread queue (I/O ‖ GPU ‖ CPU)	**Task / pipeline parallelism**
Within GPU stages (S2, S3)	OpenCL work-items over rows, columns, elements	Data parallelism (fine-grained)
Within CPU stage (S4)	OpenMP `parallel for` reduction	Data parallelism (fine-grained)
Stage-to-device assignment	CPU for S1/S4, GPU for S2/S3	**Functional decomposition**

Listing 3.3 shows the fine-grained mechanisms: OpenCL work-items that each transform one row or column, and an OpenMP loop whose threads each write only their own output slot, so no locking is required.

```
/* S2 on GPU: 2D FFT in two passes; one work-item per row, then per column */
__kernel void fft_magnitude(__global const float* in, __global float* scratch,
                            __global float* out, int width, int height,
                            int channels, int pass) {
    float re[MAX_FFT_LEN], im[MAX_FFT_LEN];
    int gid = get_global_id(0), len = (pass == 0) ? height : width;
    int line = gid % len, c = (gid / len) % channels;
    /* pass 0: real row of 'in' -> fft_1d -> complex row of 'scratch'
       pass 1: complex column of 'scratch' -> fft_1d -> out = sqrt(re^2 + im^2)
    */
}
/* S4 on CPU: images of one batch split across OpenMP threads */
#pragma omp parallel for schedule(static)
for (int i = 0; i < batch_size; ++i) {
    const float *row = processed_data + (size_t)i * ppi;
    double sum = 0.0;
    for (size_t j = 0; j < ppi; ++j) sum += row[j];
    features_out[i] = (float)(sum / (double)ppi); /* own slot */
}
```

Listing 3.3: Fine-grained data parallelism on the GPU (OpenCL) and CPU (OpenMP), simplified from `src/pipeline_kernels.cpp`.

Figure 3.5 contrasts the two patterns: only (B), in which distinct operations overlap in time on different data, is task parallelism; (A) alone, however many ranks are used, remains data parallelism.

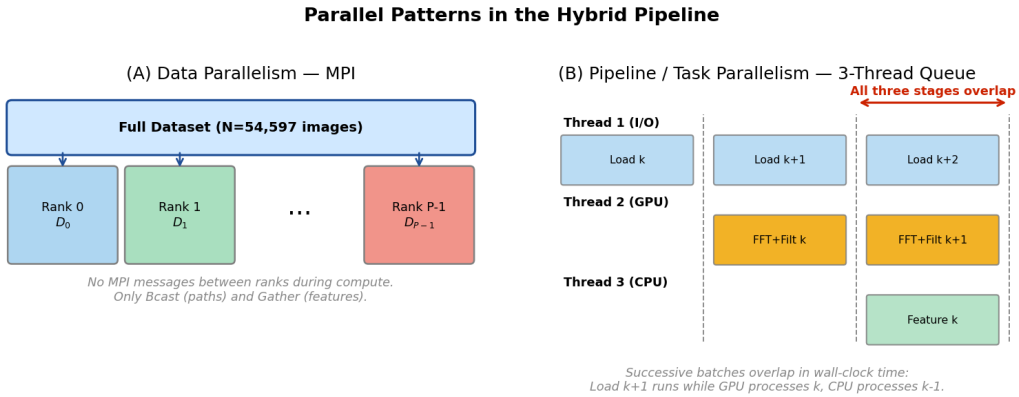


Figure 3.5: The two parallelism patterns of the hybrid pipeline: (A) MPI data parallelism across ranks, and (B) three-thread task parallelism across successive batches within one rank.

3.4 User-Customisable Workflow

A YAML-driven Directed Acyclic Graph (DAG) engine (`src/pipeline/task_graph.py`) allows users to reconfigure the pipeline at runtime without recompilation. Stages declare the data they read and write (pixels, spectrum, filtered, features); dependencies between these stages are validated on every rank using Kahn's topological sort before processing begins. Users can enable, disable, or reorder stages in `config/pipeline.yaml`. Missing inputs or unknown stage names cause a configuration error, while unused outputs generate a warning. The three-thread pipeline requires the default stage order; other valid stage orders use the sequential DAG scheduler. Listing 3.4 shows the main configuration options, and Figure 3.6 illustrates the execution model.

```yaml
dataset:
  root: "data/EuroSAT"
  target_width: 64
  target_height: 64
pipeline:
  channels: 3
  stages:
    - io
    - fft
    - filter
    - feature
  task_parallel_prefetch: true
  io_batch_size: 256
```

Listing 3.4: Workflow configuration (excerpt of `config/pipeline.yaml`).

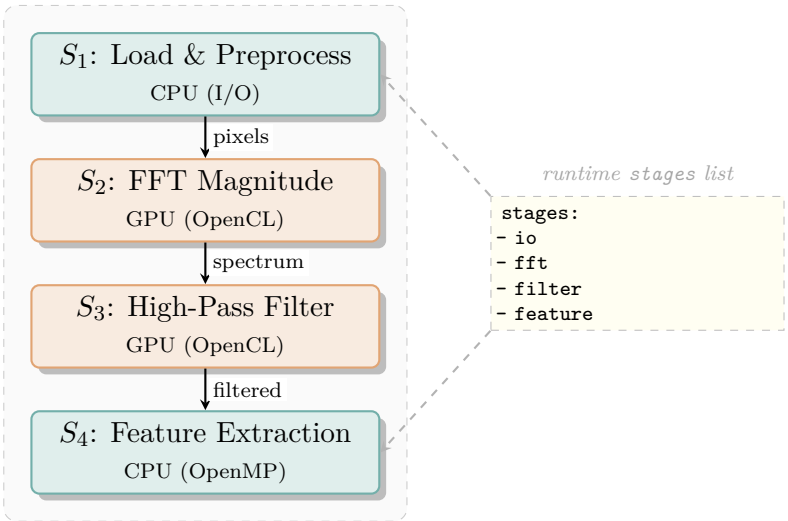


Figure 3.6: DAG execution model. Each node is a stage constructed at runtime; edges carry the data passed between stages (teal = CPU, orange = GPU). Reordering or removing entries in the YAML `stages` list (inset) rebuilds and re-validates the DAG without recompilation.

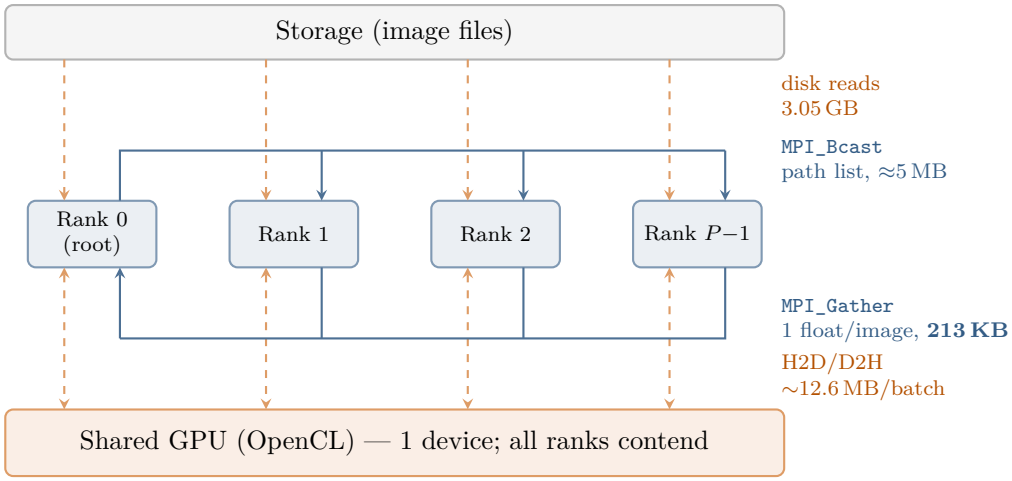
CHAPTER 4

Communication, Aggregation, and Agglomeration

This chapter covers the second and third PCAM steps: communication, including the aggregation of results, and agglomeration.

4.1 Interaction Graph and Communication Costs

Figure 4.1 shows the communicating entities and Table 4.1 the volume and cost of each communication.



Legend. Solid blue: MPI collectives (`MPI_Bcast`, `MPI_Gather`). Orange dashed: disk reads and OpenCL host–device transfers (H2D/D2H). The timing barrier and the gather of per-rank times are not drawn.

Figure 4.1: Interaction graph. All MPI ranks share one physical GPU.

Table 4.1: Communication and interaction costs for the full EuroSAT dataset (N = 54,597, $P = 8$).

Operation	Type	Volume	Run-time share
Path list `MPI_Bcast`	MPI collective	≈5 MB	<1%
Image loading	Disk I/O	3.05 GB read	≈98%
GPU-stage dispatch† (H2D + kernel + D2H + API call)	OpenCL	∼12.6 MB/batch	≈12%
Feature `MPI_Gather`	MPI collective	**213 KB**	<0.1%
Per-rank time `MPI_Gather`	MPI collective	1 float/rank	<0.1%
`MPI_Barrier` (timing)	MPI collective	0 B	<0.1%

†Run-time shares for image loading and GPU-stage dispatch are the mean busy times of the GeoTIFF ranks at P=8 from Table 5.7 (S_1 versus S_2+S_3 combined), as a fraction of their wall time; the stages overlap, so the shares do not sum to 100 %. MPI shares are measured (`experiments/measure_mpi_collectives.py`): about 9 ms for the broadcast and under 0.1 ms for the others, against $T_8 = 5.37$ s.

4.2 Feature Aggregation Strategy

Gathering raw pixel data after processing would require transferring $54{,}597 \times 64 \times 64 \times 3 \times 4\,\text{B} = 2.68\,\text{GB}$ over MPI, a serialisation bottleneck at rank 0. Instead, each rank reduces every image to one scalar (Equation (2.4)), so the gather carries $N \times 4\,\text{B} = 213\,\text{KB}$, a reduction of more than 12,000×.

4.3 Agglomeration

Agglomeration groups the fine-grained units of work (one image, one array element) into coarser ones sized to amortise fixed per-unit overheads. Unlike the *aggregation* of Section 4.2, which combines computed results, it groups *work* before it is computed. Two decisions are made.

Data agglomeration: contiguous per-rank blocks. Each rank's share (Equation (3.1)) is one contiguous range of the file list, not a cyclic distribution. This balances image *counts*, not *cost*: the list, in directory-walk order, holds 14,000 JPEGs, then the 27,597 GeoTIFFs, which take more than twice as long to load, then 13,000 JPEGs. At $P = 2$ each rank gets an even mix, but from $P = 4$ the middle ranks hold mainly GeoTIFFs and finish last (Section 5.6.2).

Task agglomeration: fixed-size batches. Within each rank, images are grouped into batches of `io_batch_size` images (default 256), which trades off two costs:

- **Larger batches** amortise the fixed cost of each GPU dispatch (Section 5.5.3). A batch size of 256 gives 214 dispatches per stage at $P = 1$ and 216 in total

at $P = 8$ (27 per rank), against 854 and 856 at a batch size of 64; each FFT dispatch runs its two passes.

- **Smaller batches** limit in-flight memory. A batch is ≈12.6 MB of pixels, and ≈38 MB once it carries its spectrum and filtered arrays. An approximate upper bound on the image data in flight per rank is therefore ≈240 MB: both `queue.Queue(maxsize=3)` queues full (3 batches of 12.6 MB and 3 of 38 MB) plus one batch held by each of the three worker threads. This estimate excludes the interpreter, the libraries and the FFT's temporary buffers; a $P = 1$ run's measured peak resident memory, interpreter included, is 116–153 MB.

The value 256 is a middle ground between the two, not exhaustively tuned. Table 4.2 summarises these decisions.

Table 4.2: Agglomeration and mapping summary. Rows 1–2 are the agglomeration decisions above; rows 3–5 anticipate the fourth PCAM step, the mapping of Section 5.1.

Decision	**Strategy chosen**	**Rationale**	**Granularity**
Data agglomeration (per rank)	Contiguous block (Eq. (3.1))	Equal image counts; simplest closed-form split; same images per rank on every run (JPEG/GeoTIFF cost differs, §5.6.2)	≈6,825 img/rank at P=8
Task agglomeration (per batch)	Fixed batch, `io_batch_size`	Amortises the fixed GPU dispatch cost (§5.5.3) vs. pipeline latency/memory	256 img/batch (≈12.6 MB)
Mapping: ranks → processes	1 MPI rank = 1 OS process	Standard MPI execution model	$P \in \{1, 2, 4, 8\}$ measured
Mapping: ranks → GPU	`device_idx = rank % num_devices`	Shares the one available GPU here; multi-GPU needs no code change	1 shared GPU (this testbed)
Mapping: OpenMP → cores	`#pragma omp parallel for` in Stage 4	OS-scheduled across available CPU cores	Up to 10 cores (4P + 6E)

CHAPTER 5

Performance Evaluation

5.1 Testbed and Hardware Mapping

This section completes the design with the fourth PCAM step, mapping, and describes the hardware on which the experiments run.

5.1.1 Testbed Hardware

- **Machine:** MacBook Air 13-inch, M4, 2025 (fanless laptop; model identifier Mac16,12, model number MC6T4T/A).
- **Processor:** Apple M4 (ARM64; 4 performance cores + 6 efficiency cores, 10 cores total).
- **GPU: 1 integrated 8-core GPU** (unified memory architecture; no discrete PCIe GPU).
- **Memory:** 16 GB Unified RAM (120 GB/s bandwidth: manufacturer specification [3], not measured here).
- **Storage:** 256 GB Apple NVMe SSD (AP0256Z).
- **Software:** macOS 26.6.2, Open MPI 5.0.8, mpi4py 4.1.1, Python 3.12.12, Apple Clang 21.

This SoC has one physical GPU, so GPU count cannot be varied here. The runtime selects devices round-robin (`device_idx = mpi_rank % num_devices`), so a multi-GPU node needs no code change; Section 5.5 instead sweeps concurrent demand on the one GPU.

5.1.2 Hierarchical Mapping Strategy

The mapping of software entities to hardware resources follows a four-level hierarchy, illustrated in Figure 5.1.

1. **MPI ranks → OS processes.** Each MPI rank is an independent OS process scheduled onto available CPU cores by macOS. The operating system does not guarantee hard CPU-core pinning, so this is scheduling rather than fixed assignment. Core type matters: a process confined to the efficiency cores (`taskpolicy -c background`) was measured to decode images about 3× more slowly (Sec-

tion 5.3.1).

2. **MPI ranks → OpenCL command queues.** Each rank creates a private OpenCL command queue on the shared GPU through `init_opencl(rank)`. All ranks share the same physical GPU.

3. **OpenMP threads → CPU cores.** Stage 4 distributes its per-image mean reduction across CPU cores using `#pragma omp parallel for`.

4. **OpenCL work-items → GPU execution.** Stages 2 and 3 launch GPU work-items that run in parallel: one per row and then per column of each channel in Stage 2 (Listing 3.3), and one per array element in Stage 3.

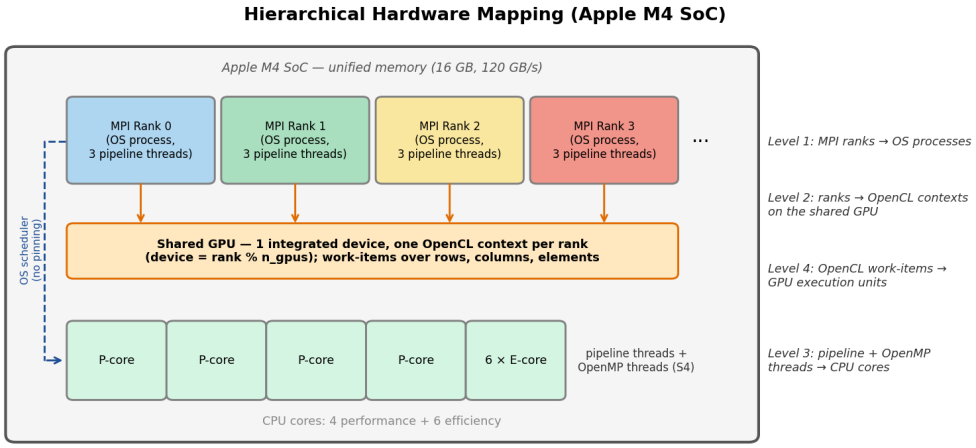


Figure 5.1: Hierarchical mapping of software entities to hardware resources.

5.2 Experimental Methodology

Each rank's wall-clock time runs from a barrier after OpenCL initialisation to the end of its local pipeline, and scaling tables use the maximum across ranks. Unless stated otherwise, experiments use the three-thread pipeline. Each timed run follows an untimed run of the same configuration, and values are averages of repeated runs interleaved across configurations.

5.2.1 File-Cache State and Experimental Repeatability

Execution times depend strongly on the operating system's file cache. When the 3.05 GB dataset is mostly cached, a $P = 1$ run takes approximately 6.3 s. When images must be read from disk, the run takes approximately 16 s and involves around 3 GB of disk reads. Most full-dataset experiments therefore used a 7 GB memory-holding process (`experiments/hold_memory.py`) to encourage disk-based reading. However, monitoring with `iostat` showed that not all experiments ran under identical cache conditions: strong scaling, the sequential comparison, the stage breakdown, the partition comparison and GPU concurrency at $P = 8$ were predominantly disk-based, while

weak scaling and the other smaller experiments were mostly cached. The stage-overlap timing and the pipeline on/off comparison ran without held memory (Section 5.6.4). GPU-stage timing and the OpenMP FFT sweep did not involve dataset file access.

Repeat sessions reproduced the main qualitative findings, but absolute execution times varied with cache state. For example, a session whose first run measured $T_1 = 11.9\,\text{s}$ fell to 6.3 s as held memory was swapped out and the dataset became cached; this session was discarded. A subsequent session with 9 GB of actively held memory (`hold_memory.py 9 --active`, 5 rounds) was more stable, giving $T_1 = 13.05\,\text{s}$, $T_8 = 5.96\,\text{s}$, $S_2 = 1.77$, $S_4 = 2.01$ and $S_8 = 2.19$. With the dataset cached (3 rounds), $T_1 = 6.80\,\text{s}$, $T_8 = 2.64\,\text{s}$, $S_2 = 1.79$, $S_4 = 2.03$ and $S_8 = 2.58$ (`results/reproduction_20261103/`). Across the reported sessions, the measured S_8 ranged from approximately 2.2 to 3.2, so absolute timings must be read with their cache state. Nevertheless, the main findings remained consistent: image loading was the major bottleneck, load imbalance appeared from $P = 4$, the OpenMP pipeline sweep showed little improvement, and cyclic partitioning improved performance.

5.3 Strong Scaling

5.3.1 Superlinear vs. Sublinear Speedup

Three mechanisms can in principle produce speedup above the ideal $S_P = P$: smaller per-rank working sets that fit in cache, better GPU occupancy when several ranks share it, and measurement artefacts that inflate T_1. None is observed: Table 5.1 shows $S_8 \approx 3.17$, and the speedup is sublinear from $P = 2$ onwards.

At $P = 2$ scaling is near-linear ($S_2 = 1.86$, efficiency 92.9 %), because each rank receives an even mix of JPEGs and GeoTIFFs, so the two ranks finish close together (8.19 s and 9.17 s). From $P = 4$ the dominant limit is load imbalance: the contiguous partition (Section 4.3) gives the outer ranks mainly JPEGs and the middle ranks mainly GeoTIFFs, whose Load stage takes more than twice as long. The "Fastest rank" column of Table 5.1 shows the JPEG ranks finishing at about half of T_P or earlier and then waiting. Section 5.6.2 isolates the effect. The secondary effect is the shared GPU: all ranks submit to one device (Section 5.5.3). The CPU is shared too: eight ranks of three threads each run on ten cores, only four of them performance cores.

5.3.2 Results

Table 5.1: Strong-scaling results for the full EuroSAT dataset ($N = 54{,}597$ images): average of 5 interleaved rounds (`results/strong_scaling.csv`). Speedup $S_P = T_1/T_P$; efficiency $E_P = S_P/P$; "Fastest rank" is the mean time of the quickest rank, against T_P for the slowest.

P	Images/rank	T_P [s]	Fastest rank [s]	S_P	E_P	Throughput [img/s]
1	54,597	17.03	17.03	1.00	100.0%	3,206
2	27,299	9.17	8.19	1.86	92.9%	5,956
4	13,650	7.06	3.16	2.41	60.3%	7,732
8	6,825	5.37	2.60	3.17	39.7%	**10,170**

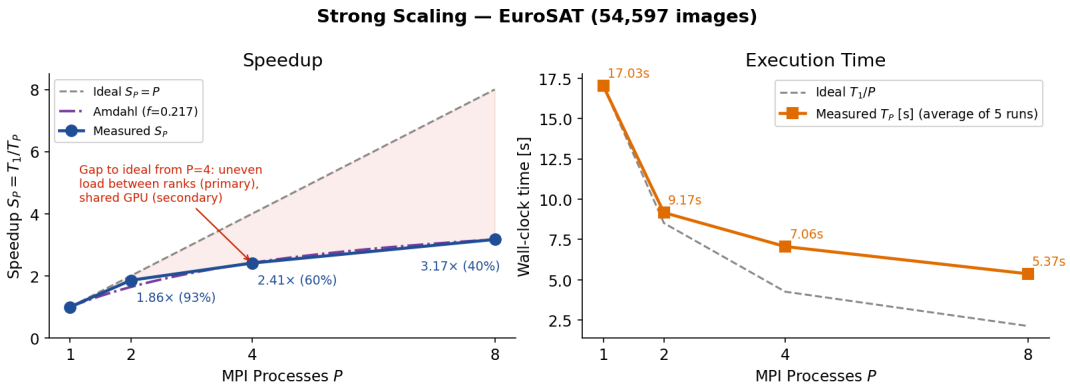


Figure 5.2: Strong-scaling speedup against the ideal $S_P = P$ and the Amdahl's Law fit of Section 5.3.3.

5.3.3 Theoretical Bound: Amdahl's Law

Amdahl's Law [2] bounds the speedup of a fixed-size problem on P processing elements when a fraction f of the work does not shrink with P:

$$S_P = \frac{1}{f + \dfrac{1-f}{P}}, \qquad S_\infty = \lim_{P\to\infty} S_P = \frac{1}{f}. \tag{5.1}$$

Solving Equation (5.1) for f using the $P = 8$ measurement in Table 5.1 ($S_8 = 3.17$) gives

$$f = \left.\frac{P/S_P - 1}{P - 1}\right|_{P=8} \approx 0.217 \quad (22\,\%),$$

implying a ceiling of $S_\infty = 1/f \approx 4.6\times$ on this testbed with the current partitioning, untested since the testbed never ran $P > 8$. Because f is fitted from one point, moving it by 0.06 either way gives ceilings of 6.35× and 3.60× (Figure 5.3).

Table 5.2: Measured speedup versus Amdahl's Law, $f = 0.217$ fitted at $P = 8$, and the Karp–Flatt metric e (`experiments/analyze_amdahl.py`).

P	**Measured** S_P	**Amdahl** S_P	**Difference**	**Karp–Flatt** e
1	1.00	1.00	0.0 %	—
2	1.86	1.64	+13.1 %	0.076
4	2.41	2.42	−0.4 %	0.220
8	3.17	3.17	0.0 % (fit point)	0.217

Table 5.2 compares the fit with the measurements. It is within 1 % at $P = 4$ and exact at $P = 8$ by construction, but underestimates the speedup at $P = 2$ by 13.1 %, where the ranks are balanced and the load imbalance that dominates f has not yet appeared. f is therefore not a literal serial-code fraction but an effective non-scaling fraction, dependent on P.

Karp–Flatt metric. The experimentally determined serial fraction [11], $e = (1/S_P - 1/P)/(1 - 1/P)$, tests whether f really is constant. At $P = 8$ it equals the fitted f by construction. It rises from 0.076 at $P = 2$ to 0.220 at $P = 4$ and then stays flat at 0.217: the non-scaling fraction is not a fixed serial part of the code but appears when the load imbalance does. In the repeat session of Section 5.2 the fit gave $f = 0.38$ and a ceiling of 2.6×, with e of 0.128, 0.330 and 0.379: the same pattern, at a different level. The ceiling is therefore not a fixed property of the code. The symbol f alone denotes this fraction; f_i in Chapter 2 is a per-image feature.

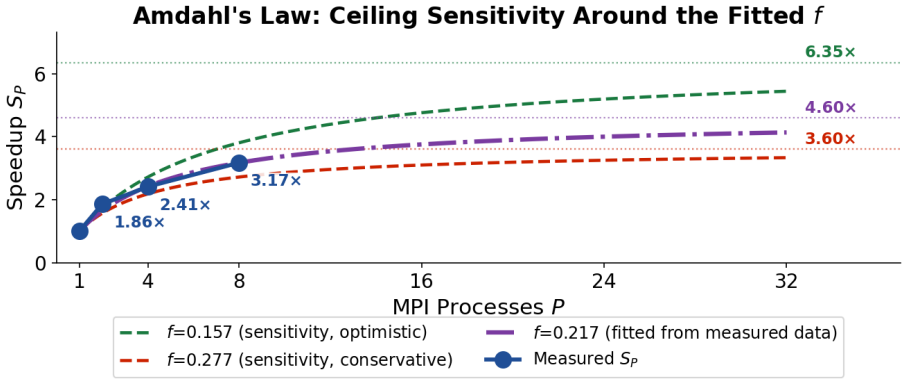


Figure 5.3: Sensitivity of the Amdahl's Law ceiling to the fitted effective non-scaling fraction. The thick curve ($f = 0.217$) is the fit of Figure 5.2; the dashed curves ($f = 0.157$, $f = 0.277$) are hypothetical, and all curves beyond $P = 8$ are projections.

5.4 Weak Scaling

Weak scaling holds the workload at 1,000 images per rank. Rank r processes files $[1000r, 1000(r+1))$, all JPEGs for $P \leq 8$, and efficiency is $E_P^{\text{weak}} = T_1/T_P$. Table 5.3 and Figure 5.4 give the results.

Table 5.3: Weak-scaling results (1,000 images per rank): 4 interleaved rounds, averages; raw data in `results/weak_scaling_raw_trials.csv`.

P	**Total images**	T_P [s]	T_P/T_1	E_P^{weak}
1	1,000	0.180	1.00	100.0%
2	2,000	0.188	1.05	95.5%
4	4,000	0.258	1.43	69.7%
8	8,000	0.383	2.13	46.9%

Efficiency declines from 95.5 % at $P = 2$ to 46.9 % at $P = 8$, as growing total work increases demand on the shared CPU cores and GPU. Every rank processes only JPEGs, so this decline is not the JPEG/GeoTIFF imbalance. The runs varied by under 9 % across the 4 rounds.

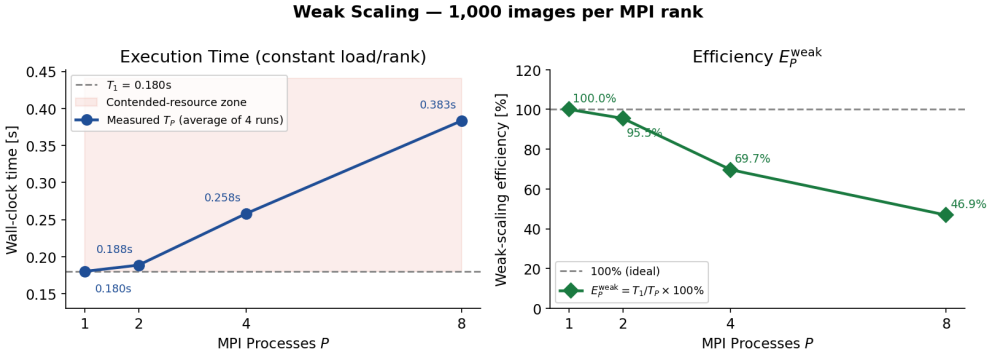


Figure 5.4: Weak scaling: wall-clock time stays near T_1 through $P = 2$, then grows with P.

5.5 GPU Concurrency Experiment

5.5.1 Experimental Design

The experiment (`experiments/run_gpu_scaling.py`, optionally with `--reverse`) sweeps $P \in \{1, 2, 4, 8\}$ ranks, each processing 5,000 images, to quantify how concurrent demand on the shared GPU affects throughput. Rank r takes files $[5000r, 5000(r{+}1))$ of the file list, so both ranks process JPEGs at $P = 2$, while from $P = 4$ later ranks receive the costlier GeoTIFFs (Section 5.6.2): rank 2 holds 1,000 of them, and rank 3 at $P = 4$ and ranks 3–7 at $P = 8$ hold only GeoTIFFs.

Each P is measured across four sweeps, one in reverse rank order to expose drift, with a coefficient of variation of 2–10 % (raw data: `results/gpu_scaling_raw_sweeps.csv`).

5.5.2 Results and Analysis

Table 5.4 and Figure 5.5 give the results.

Table 5.4: GPU concurrency experiment: 5,000 images/rank, mean of 4 independent sweeps with 7 GB held, all ranks sharing one integrated GPU. Throughput is the mean over the sweeps; T_P is total images divided by that mean. The last column is a repeat without held memory (2 sweeps), with the images cached.

P	**Total images**	T_P [s]	**Throughput [img/s]**	S_P^{thr}	**Cached [img/s]**
1	5,000	0.484	10,325	1.00×	10,222
2	10,000	0.685	14,606	1.41×	17,677
4	20,000	1.852	10,800	1.05×	14,257
8	40,000	4.712	8,489	0.82×	18,860

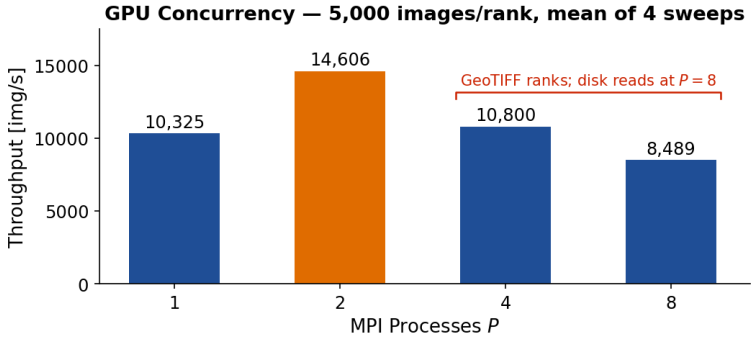


Figure 5.5: GPU concurrency experiment: aggregate throughput peaks at $P = 2$, where both ranks process JPEGs, and falls from $P = 4$, where GeoTIFF ranks set the time and, at $P = 8$, images are read from disk.

Throughput peaks at $P = 2$ (1.41×), where two JPEG ranks share the GPU with contention for the device and the CPU cores. It falls at $P = 4$ (1.05×) and $P = 8$ (0.82×) as per-rank times spread out: in the final run of the last sweep they were 0.74–0.76 s at $P = 2$ (T_2 varied from 0.63 s to 0.80 s across sweeps), 0.97–1.89 s at $P = 4$ and 1.45–4.34 s at $P = 8$. The pattern held in all four sweeps. The repeat with the images cached shows how much of the fall is disk access: throughput at $P = 8$ is 18,860 images/s instead of 8,489, so that fall is mainly reading from disk rather than GPU contention. The dip at $P = 4$ remains, with ranks spreading from 0.75 s to 1.42 s, which is the load imbalance of Section 5.6.2. A multi-GPU node would remove the competition for one device; this was not measured.

5.5.3 Verification of GPU Utilisation

`init_opencl()` requests `CL_DEVICE_TYPE_GPU` and falls back to the CPU only if no GPU is found; every run prints `[CPP] Rank 0 -> GPU[0] Apple M4`. An independent script using PyOpenCL (`experiments/verify_gpu_profiling.py`), which reads the production kernel source but bypasses the C++ library, confirms non-zero driver-recorded device execution for all 214 batches. These driver timestamps are used only as evidence of execution, not for timing: they report about 0.03 ms for a 64-image kernel run that the host clock measures at about 2 ms (`experiments/verify_gpu_`

`breakdown.py`), and only the host clock grows with the batch size as the work does. That script's 2 ms and the 3.05 ms in Table 5.5 differ through run-to-run GPU state rather than method: alternating the two in one session gave 2.07–2.47 ms and 1.73–4.29 ms respectively.

`experiments/run_gpu_stage_timing.py` times the production calls with the host clock, with `clFinish` after each phase (25 trials per batch size after 5 discarded), and the same FFT on the CPU.

Table 5.5: Time per call [ms] of the GPU stages and of the same FFT on the CPU, host clock, random $64 \times 64 \times 3$ images.

Batch	FFT kernel	FFT call	Filter call	NumPy FFT	OpenMP FFT
16	1.78	2.31	0.76	1.53	0.81
64	3.05	3.49	0.89	6.91	3.04
256	4.91	5.61	2.14	25.39	11.65
1024	17.13	18.78	7.46	103.08	44.61

Most of the FFT call is kernel execution, and the time grows with the batch. A straight-line fit gives a fixed cost per FFT dispatch of about 2.0 ms plus 16 μs per image here, which is 36 % of the call at the production batch of 256. Three repeats of the sweep gave a smaller fixed cost, 0.9–1.3 ms (16–23 % at 256), so this cost varies with GPU state, and the per-image cost of 16–18 μs is the stable figure. Larger batches amortise the fixed cost (Section 4.3). At a batch of 256 the GPU FFT is about 4.5× faster than single-threaded NumPy and 2.1× faster than the ten-thread OpenMP path, and the call time reproduced within 4 % (5.6–5.8 ms). At a batch of 16 the OpenMP path is faster and NumPy comparable to the GPU. GPU speed depends on its power state: in one earlier session the 256-image call took 15.5 ms, slower than OpenMP (11.8 ms) but still faster than NumPy (27.4 ms).

Offloading is justified for this batch size, but at $P = 8$ the two GPU stages occupy only about 12 % of a GeoTIFF rank's time (Table 5.7; 19 % in the repeat session), so a multi-GPU deployment would relieve contention without removing the larger limit, the Load stage.

5.6 Additional Experiments

5.6.1 OpenMP Thread Scaling

With a single MPI rank, `OMP_NUM_THREADS` is varied via `experiments/run_openmp_scaling.py` while the whole single-rank pipeline runs end to end on the first 20,000 files (14,000 JPEGs, 6,000 GeoTIFFs), in three interleaved rounds (Table 5.6, Figure 5.6).

Table 5.6: OpenMP thread scaling ($P = 1$, 20,000 images, mean of 3 interleaved rounds).

Threads	T [s]	$S_t = T_1/T$	**Throughput [img/s]**
1	2.131	1.00	9,384
2	2.159	0.99	9,263
4	2.141	1.00	9,343
8	2.156	0.99	9,276

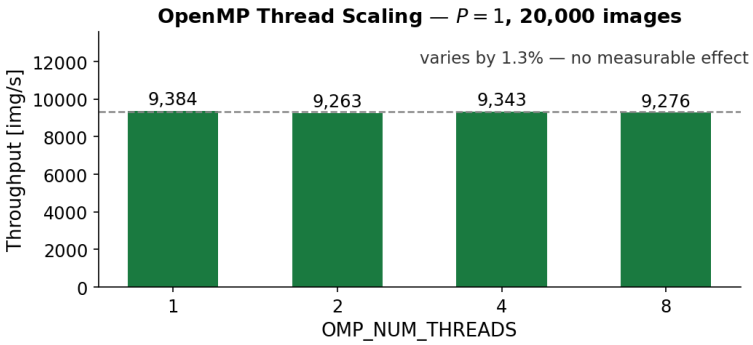


Figure 5.6: OpenMP thread scaling: throughput varies by less than 2 % across 1–8 threads.

Thread count has no measurable effect on end-to-end pipeline time. This follows from which code path runs: when a GPU is available, Stages 2–3 execute via the OpenCL kernels rather than the OpenMP CPU-fallback paths (`cpu_fft_magnitude`, `cpu_highpass`) also present in `pipeline_kernels.cpp`. The *only* OpenMP-parallel code on the hot path is Stage 4's `extract_features_cpu`, which was measured at 45 ms on these 20,000 images – about 2 % of the ≈2.1 s total, too small a share for any thread count to produce a measurable effect. OpenMP matters on a GPU-less deployment, where Stages 2–3 fall back to the CPU paths above. Sweeping the OpenMP FFT path itself (`experiments/run_openmp_fft_scaling.py`, 1,024 random images, 5 interleaved rounds) gives 218.6, 112.0, 61.9, 46.9 and 43.4 ms at 1, 2, 4, 8 and 10 threads: speedups of 1.95, 3.53, 4.67 and 5.03× (a repeat reached 4.64× at 10 threads), with the gain flattening beyond the 4 performance cores.

5.6.2 Pipeline Stage Time Breakdown

Table 5.7: Per-stage busy time at $P = 8$ ($N = 54{,}597$), measured by timing the production stage functions under `mpirun -n 8` (`experiments/run_stage_breakdown.py`), average of 5 trials, for the JPEG ranks (0, 1, 7; 6,824–6,825 JPEGs each) and the GeoTIFF ranks (3, 4, 5; 6,824–6,825 GeoTIFFs each); ranks 2 and 6 hold a mix. Stages overlap in the pipeline, so busy times do not add up to the wall time.

Stage	JPEG rank [s]	GeoTIFF rank [s]	Device
S_1: Load & Preprocess	2.41	**5.65**	CPU
S_2: FFT magnitude	0.44	0.52	GPU (OpenCL)
S_3: High-pass filter	0.19	0.20	GPU (OpenCL)
S_4: Feature extraction	0.04	0.05	CPU (OpenMP)
Rank wall time	2.56	**5.79**	

The ranks hold the same number of images, yet the GeoTIFF ranks take more than twice as long, and almost the whole difference lies in the Load stage: a 13-band GeoTIFF is about 31× larger on disk than a JPEG and is decoded with `tifffile` rather than Pillow, while the GPU and feature stages cost nearly the same for both. The slowest rank is a GeoTIFF rank in every trial, and on the GeoTIFF ranks Load is busy for 98 % of the wall time (GPU stages 9 % and 3 %, features 1 %), so Load is the bottleneck stage. Replacing the contiguous split by a cyclic one (`all_files[rank::size]`), which gives every rank the same JPEG/GeoTIFF mix, was measured with the production pipeline patched from outside (`experiments/run_partition_comparison.py`, 3 alternating trials, Table 5.8): it reduced T_P by 35 % at $P = 4$ and 28 % at $P = 8$ (−38 % and −35 % in the repeat session), and in all six cyclic runs the fastest and slowest ranks finished within 0.13 s of each other. The contiguous times differ from Table 5.1, measured in a separate session (Section 5.2). $P = 2$ was not measured, since its ranks are already close.

Table 5.8: Contiguous versus cyclic partition, mean of 3 alternating trials (`results/partition_comparison.csv`). The first column repeats Table 5.1 (a separate session) for reference.

P	Table 5.1 [s]	Contiguous [s]	Cyclic [s]	Change
4	7.06	7.91	5.14	−35 %
8	5.37	5.70	4.10	−28 %

The contiguous split remains the production design because the cyclic split was measured after all other experiments, which use the contiguous split; changing it would have broken their comparability. Adopting the cyclic split is the first item of future work.

5.6.3 Pipeline Stage-Overlap Verification

Table 5.9 presents measured per-batch stage start/end times recorded by `experiments/run_pipeline_timing.py` in pipeline mode (512 images, batch size 64, single rank). Batch 1 Load (4.14–8.43 ms) overlaps with Batch 0 GPU execution (4.15–7.11 ms) for their entire common window, and Batch 0 CPU (7.13–7.38 ms) falls inside the still-running Batch 1 Load interval. The table is one representative run; stage times vary (Batch 0 GPU took 5.3 ms in a repeat), but the script's `[VERIFIED] Pipeline overlap` line confirmed the Load/GPU overlap in every run. The GPU stage here (2.96 ms) is shorter than the 4.38 ms of Table 5.5 at the same batch size (3.49 + 0.89 ms): a different session, within the range of GPU-state variation (1.73–4.29 ms, Section 5.5.3).

Table 5.9: Measured pipeline timing ($P = 1$, batch size 64, relative wall-clock milliseconds).

Batch	Stage	Start [ms]	End [ms]	Duration [ms]	Overlap?
0	Load	0.06	4.14	4.08	—
0	GPU	4.15	7.11	2.96	—
1	Load	4.14	8.43	4.29	**Yes (with Batch 0 GPU)**
0	CPU	7.13	7.38	0.26	**Yes (with Batch 1 Load)**
1	GPU	8.44	11.54	3.10	**Yes (with Batch 2 Load)**
2	Load	8.44	12.73	4.29	**Yes (with Batch 1 GPU)**

5.6.4 Task-Parallelism Payoff: Pipeline Mode vs. Sequential Mode

Pipeline mode is compared end to end with sequential mode (Section 3.3) on the full dataset at $P = 1$ and $P = 8$. Three rounds alternate the two modes, each timed run preceded by an untimed run of the same mode, without held memory (`experiments/run_pipeline_onoff_comparison.py`).

Table 5.10: Pipeline mode versus sequential mode, full dataset, mean of 3 alternating rounds.

P	Pipeline ON [s]	Pipeline OFF [s]	Speedup
1	6.32	22.04	3.49×
8	2.59	5.99	2.32×

Pipeline mode is faster than sequential mode in every one of the twelve timed runs; a repeat of the experiment gave 3.37× and 2.51×. Two mechanisms contribute. The first is stage overlap: overlap alone bounds the speedup of a pipeline limited by its slowest stage at $(t_{\text{load}} + t_{\text{gpu}} + t_{\text{cpu}})/\max(t_{\text{load}}, t_{\text{gpu}}, t_{\text{cpu}})$, which is about 1.8× with the stage times of Table 5.9 and 1.9× in the repeat of that measurement. The second is the file cache: sequential mode materialises the rank's entire subset and its spectrum and

filtered copies (about 5 GB resident at $P = 1$), against an estimated bound of ≈240 MB per rank for the pipeline (Section 4.3). This footprint evicts the images from the OS file cache, so sequential runs read them from disk while pipeline runs keep them cached. Starting from a cached state at $P = 1$, a sequential run took 11.0 s against 6.3 s for pipeline runs (1.75×, close to the overlap bound), and the pipeline run directly after it took 17.4 s before returning to 6.3 s. Table 5.10 shows each mode's steady state.

5.7 Summary and Comparative Benchmarking

The sequential row is a single-threaded, pure NumPy implementation of Algorithm 1 (no MPI, OpenMP or OpenCL) with the project's own loader, timed in the same three rounds as the hybrid runs (`experiments/run_comparative_benchmark.py`), and the hybrid pipeline was faster in every round (Table 5.11, Figure 5.7). All rows compute the same result: the sequential program and the pipeline agree to within 10^{-6} on 3,000 random files, and the pipeline with NumPy on all files (Chapter 2).

Table 5.11: Comparative performance for $N = 54{,}597$ images: mean of 3 rounds in which all five implementations are timed (`results/comparative_benchmark.csv`).

Implementation	T [s]	**Speedup vs. sequential**
Sequential (1 thread, pure NumPy, no MPI/GPU)	22.61	1.00×
Hybrid pipeline, $P = 1$	15.86	1.43×
Hybrid pipeline, $P = 2$	9.45	2.39×
Hybrid pipeline, $P = 4$	6.95	3.25×
Hybrid pipeline, $P = 8$	**5.04**	**4.49×**

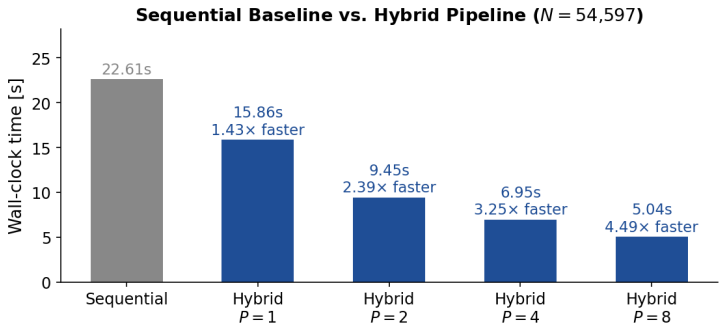


Figure 5.7: Sequential baseline versus the hybrid pipeline at each P. The hybrid pipeline is faster at every P, including $P = 1$.

At $P = 1$ the hybrid pipeline is already 1.43× faster: the sequential loop spends 69.8 % of its time loading and 26.2 % in the FFT, whereas in the pipeline Stages 2–4 overlap with loading, so the $P = 1$ time is essentially the Load stage alone (about 15.7 s of the sequential run, against 15.86 s). MPI raises the advantage to 4.49× at $P = 8$, limited by the load imbalance (Section 5.6.2); with the images cached the times are 13.5 s sequential and 6.80 s and 2.64 s at $P = 1$ and $P = 8$ (1.98× and 5.11×).

CHAPTER 6

Conclusions

6.1 Summary

This project designed, implemented, and evaluated a hybrid MPI + OpenCL + OpenMP pipeline for frequency-domain processing of 54,597 EuroSAT image files, combining data parallelism across ranks, task parallelism within each rank, and a configurable workflow with minimal communication.

Principal findings. The pipeline reaches ≈10,200 images/s at $P = 8$ ($S_8 \approx 3.17$, and 2.2–3.2 across repeat sessions; efficiency 92.9 % at $P = 2$), 1.43×/4.49× faster than the sequential baseline at $P = 1/P = 8$, with features matching a NumPy reference; pipelined execution is 2.3× faster than sequential-stage execution at $P = 8$, with stage overlap explaining at most about 1.9× and a file-cache effect the rest. Scaling is limited mainly by load imbalance from $P = 4$ (mainly-JPEG versus mainly-GeoTIFF ranks) and secondarily by the shared GPU; the Load stage, not the GPU, is the bottleneck. OpenMP has no measurable end-to-end effect while the GPU runs Stages 2–3, because the only OpenMP stage on that path takes about 2 % of the time, but on the CPU FFT path used without a GPU it gives about 5× at 10 threads.

6.2 Limitations and Future Work

- **Load imbalance.** A cyclic split gives every rank the same JPEG/GeoTIFF mix; it reduced T_8 by 28 % when measured.
- **Single node, one GPU.** Multi-node scaling is unmeasured.
- **FFT kernel.** It handles power-of-two sizes in single precision; batched libraries such as clFFT [1] offer more.
- **GPU dispatch cost.** Persistent buffers could reduce the fixed cost per dispatch.
- **Laptop testbed.** Timings and speedups depend on the OS file cache, which is only partly controlled here.

References

[1] AMD (2014). *clFFT: OpenCL Fast Fourier Transform Library*. AMD Open Source. https://github.com/clMathLibraries/clFFT

[2] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485.

[3] Apple (2025). *MacBook Air (13-inch, M4, 2025) – Tech Specs*. https://support.apple.com/en-us/122209

[4] Cooley, J. W. and Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301.

[5] Drusch, M. et al. (2012). Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote Sensing of Environment*, 120:25–36.

[6] Foster, I. (1995). *Designing and Building Parallel Programs*. Addison-Wesley.

[7] Frigo, M. and Johnson, S. G. (2005). The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231.

[8] Frigo, M. and Johnson, S. G. *FFTW 3 Manual: Distributed-memory FFTW with MPI*. https://www.fftw.org/fftw3_doc/Distributed_002dmemory-FFTW-with-MPI.html

[9] Gorelick, N. et al. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202:18–27.

[10] Helber, P. et al. (2019). EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226.

[11] Karp, A. H. and Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5):539–543.

[12] MPI Forum (2021). *MPI: A Message-Passing Interface Standard, Version 4.0*. Technical report. https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf

[13] OpenMP Architecture Review Board (2018). *OpenMP Application Programming Interface, Version 5.0*. https://www.openmp.org/specifications/

[14] Rauber, T. and Rünger, G. (2013). *Parallel Programming: For Multicore and Cluster Systems*. 2nd edition. Springer, Berlin.

[15] Stone, J. E., Gohara, D., and Shi, G. (2010). OpenCL: A parallel programming standard for heterogeneous computing systems. *Computing in Science & Engineering*, 12(3):66–73.